

RAI: Safe Pipelined Epoch Transitions for Account-Local Payments

Anonymous Author(s)
Regular paper

Abstract—Single-writer payments finalize from account-local quorums without a global transaction sequence. Replacing the validators that issue those quorums is the hard part: votes already released by the outgoing committee can still form a final certificate after the handoff. RAI is an epoch-transition protocol that lets the next epoch’s account voting proceed while the previous epoch closes. Each outgoing validator freezes one signed report; RAI defines which report selections are eligible checkpoint inputs and a deterministic construction that turns them into a checkpoint carrying finality and origin-stamped locks. Reconstruction and agreement are abstract services, and RAI is safe and terminates given their safety and termination. A predecessor gate, a prefix-wide overlap exception, late notarization, and a no-fast-path rule for early votes keep the pipeline safe; TLA⁺ model checking exhibits the failures of three earlier rules and verifies the repaired ones. A prototype in RsNano settles nine fork and fault workloads at an offered 2,000 blocks/s with median latency near 100 ms.

Index Terms—Byzantine fault tolerance, reconfiguration, asset transfer, consensusless payments, model checking

I. INTRODUCTION

Single-writer value transfer does not require every payment to occupy a position in a global transaction sequence. An account owner orders its own updates; validators only have to prevent incompatible owner-signed histories from becoming final. The consensus-number characterization of asset transfer formalizes this distinction, and payment systems already exploit it [1], [2], [3]. The problem addressed here is what happens when the validators issuing those account-local votes are replaced.

A reconfiguration must decide which old outcomes the successor is obliged to preserve. Consider a payment for which enough outgoing validators have released signatures to form a final certificate, while network delay keeps those signatures apart, so that no checkpoint builder sees the certificate. If the checkpoint omits the payment and lets an incompatible successor history through, the delayed signatures later establish a valid old-epoch final outcome. The checkpoint was agreed, but it was not safe. We call such an outcome *latent-final*, and we extend the obligation to certificates that outgoing validators which have not yet stopped may still complete.

The obvious remedy is to stop: end account voting, wait until every certificate has settled, then agree on a snapshot. That costs a pause at every epoch boundary, lasting as long as the slowest certificate. RAI avoids it. It is a protocol for *pipelined* epoch transitions: epoch $e+1$ account voting starts at each validator’s local boundary, while epoch e is still closing, and the two are kept safe against each other by rules that

are local to the account path. The closing side freezes one signed *report* per outgoing validator, committing to a tagged view of its ledger and to the blocks it voted for outside that view; a deterministic construction turns a selection of reports into a checkpoint that preserves finality and carries obligation locks, and the successor committee, known two epochs ahead, installs it without voting on it.

RAI deliberately leaves two things abstract. Reports commit to sets by their roots; a *convergence* service makes every correct reporter’s frozen sets reconstructible at every correct validator. A *consensus* service decides one checkpoint per epoch among the candidates RAI declares eligible. RAI specifies the interfaces and the properties it needs from both and proves safety and termination assuming theirs. Any set-reconciliation mechanism and any Byzantine agreement protocol with external validity can fill the two slots; our prototype uses rateless set reconciliation and a Kudzu-style election.

The pipeline is where the subtle failures are. Three rules of an earlier version of this protocol were wrong in ways that written arguments did not catch: a report format that hid a fresh vote for a block the predecessor had retained, a lock discharge that let two validators reach different verdicts on the same evidence depending on installation order, and a fast path that counted first votes cast before their signers knew the predecessor checkpoint. TLA⁺ models of the handoff exhibit each failure in a few steps and verify the repaired rules exhaustively for six validators (Section VII).

Contributions. First, RAI specifies pipelined epoch transitions for account-local finality: lagged committees, local boundaries, a predecessor gate for fresh successor work, a prefix-wide overlap exception that lets settlement continue before the predecessor checkpoint is known, late notarization by the outgoing committee, and a no-fast-path rule for early votes. Second, it defines immutable signed reports with a report-exposure invariant that covers latent and late finality, origin-stamped lock records with matching-origin discharge, and a deterministic checkpoint construction over eligible report selections. Third, it states the convergence and consensus interfaces as explicit assumptions, proves transfer safety and conditional termination relative to them, and model-checks the handoff rules. Fourth, it reports a prototype in RsNano whose nine workload variants, with forks and with one offline or one Byzantine validator, settle to identical state on every validator while the unmodified node finishes none of the forked ones, and it measures what the pipeline costs in latency tails.

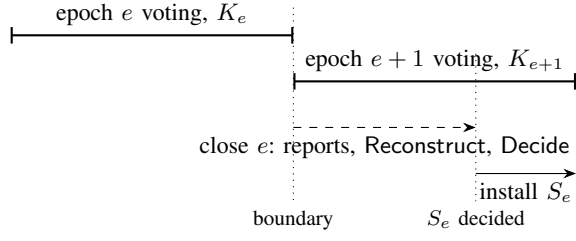


Fig. 1. A pipelined transition. Epoch $e + 1$ voting starts at the boundary; the close of e overlaps it; ordinary $e + 1$ work finalizes once S_e is installed, and the overlap exception of Section IV-C finalizes some of it earlier.

The results are protocol arguments, checked by model checking only in the finite configurations of Section VII. Section X reports single runs on one host; they illustrate behavior under load and do not validate the proofs' assumptions. The single-writer restriction and the two instantiated services are not claimed as contributions.

II. OVERVIEW

Figure 1 shows one epoch transition. Account voting for epoch e is done by committee K_e . At its local boundary a validator stops signing epoch- e first and final votes, freezes its report, and opens epoch $e + 1$ for the same accounts. The checkpoint S_e is decided by the outgoing committee from $N - f$ reports, while epoch $e + 1$ voting is already under way. The successor committee verifies and installs S_e ; only then does ordinary epoch- $(e + 1)$ work become final. Committees are lagged by two epochs: S_e selects the committee that votes in epoch $e + 2$, so the committee of $e + 1$ is known throughout the transition.

Three things must hold for this pipeline to be safe. Old-epoch finality that can still be formed must survive in S_e : the report rules and the construction of Section V guarantee it. New-epoch work must not finalize anything the old epoch could still contradict: the predecessor gate holds ordinary successor work provisional until S_e is known, the overlap exception finalizes early only where old-domain exclusion is proven at every position, and a fast certificate never counts a vote cast before its signer knew S_e . The outgoing committee supplies exclusion evidence for fresh blocks by *late notarization*: until it installs S_e , each member also notarizes in epoch e the blocks it first-votes in epoch $e + 1$, with votes that count toward no finality. And every correct validator must install the same S_e : this is where the consensus service enters, and it decides only among candidates that RAI's eligibility predicate accepts.

With $N = 6$ and $f = p = 1$, notarization and normal finality need four votes, fast finality five first votes, and recovery three reporter first votes. If validators 1, 2, 3, 4 and a Byzantine validator 6 first-vote a block B but dissemination is delayed, a candidate selecting the reports of 1, 2, 3, 5, 6 still sees $h(B)$ in the voted-but-not-ledger sets G_1, G_2, G_3 , which meet the recovery threshold, and the construction retains B under a recovery record; the five first votes can later form a

TABLE I
ACCOUNT AND CLOSURE THRESHOLDS. THE EXAMPLE HAS $f = p = 1$, $N = 6$.

Evidence	Threshold	Example
Notarization NC / exclusion witness XW	$q = N - f - p$	4
Normal finality FC	q final votes	4
Fast finality FF	$N - p$ settled first votes	5
Recovery support	$r = f + p + 1$	3
Selected reports	$N - f$	5

fast certificate that finalizes it. Every selected report quorum intersects a normal final-vote quorum in at least $p + 1$ correct signers and a fast first-vote quorum in at least $f + p + 1$: the two exposure cases of the closure theorem.

RAI fixes the report format, the eligibility of a report selection, the construction, the overlap and installation rules, and the lifecycle of epochs; it does not fix how a validator obtains another reporter's frozen sets, nor how the outgoing committee agrees on one eligible candidate (Section III-D).

III. MODEL, ASSUMPTIONS, AND ABSTRACT SERVICES

A. Participants, faults, and retained evidence

Clients own accounts and sign blocks. An authenticated committee contains $N = 3f + 2p + 1$ equal-weight validator identities, $f, p \geq 0$, of which at most f are Byzantine. Corruption is static: adversarial identities are fixed throughout the execution, and a correct departing identity is not later compromised. Membership admission, Sybil resistance, and the per-committee fault bound are deployment assumptions.

Signatures are unforgeable and hashes collision resistant; canonical encodings bind the network session and message domain. An account vote binds its epoch, committee, account, slot, vote type, block hash, and admission-witness digest, where the witness identifies the anchor and evidence under which the vote was admitted, including its base checkpoint (Section IV-B); checkpoint votes bind the handoff context and deciding committee. Distinct signatures by one identity never contribute twice to a quorum for the same block.

Correct validators persist their first vote and terminal state before releasing signatures. They retain blocks, signed votes, and finite dependency evidence, and relay them after local voting ends; a certificate is a deterministic local view of sufficient individually verified votes, not a separately gossiped object. A retained item held by a correct validator eventually reaches every correct validator that needs it (the *data service*), and departing validators continue service until correct successors hold durable copies.

B. Quorums, accounts, and canonical state

The account path uses $N = 3f + 2p + 1$, $q = 2f + p + 1$, and $r = f + p + 1$ (Table I). The parameter p is the number of

first-vote responses the fast path can omit, not an additional fault allowance. We repeatedly use

$$2q - N = f + 1, \quad q - 2f = p + 1, \quad (N - p) - 2f = r. \quad (1)$$

Each account is a tree of owner-signed blocks rooted at genesis. A block's slot is one greater than its parent's slot; slots are account positions, not times. Blocks of one account conflict when neither is an ancestor of the other. An owner equivocates by signing conflicting blocks; safety covers such owners, liveness does not promise a winner under persistent contention.

A closed checkpoint is $S_e = (L_e, \Sigma_e)$. The ledger L_e identifies finalized prefixes, retained unresolved branches, and lock records. A lock record names its block, its strength (notarization or recovery), and its *origin*: the epoch whose potential finality it protects. A block may carry several recovery records, at most one per origin. Each record is discharged separately, and only by evidence of its own origin epoch; the block stays retained while any of its records survives. The application state Σ_e contains balances, received-send identifiers, and applied-operation identifiers. A block is final either from an eligible account FC/FF or because a decided checkpoint promoted it under Section V-E; an undecided candidate has no finality effect.

Let F_e be the finalized block set of L_e . A validator also maintains a certified live overlay O_i of locally finalized blocks not yet in F_e ; its effective final set is $F_e \cup O_i$, and its live view is the deterministic replay of that set. Section V-F defines installation without rolling back or reissuing effects. A deterministic function derives the committee C_e from Σ_e , never from a private overlay.

C. Latent finality and progress assumptions

For a fixed epoch and admission context, a block is *latent-final* at a cut if released signatures and valid dependency evidence suffice to verify an eligible FC or FF for it or for a descendant selecting it; *late* finality is such a proof assembled or exposed after reports freeze or after a checkpoint is decided. A closure obligation also covers a certificate completed by subsequent old-epoch signatures from identities that have not stopped. Safety has no message-delay bound. Progress requires finite relevant admitted work, fair processing and retry, retained-data availability, and recurring sufficiently long bounded-delay windows in which δ bounds delivery and processing and clock rates lie in $[1 - \rho, 1 + \rho]$. An uncontested continuation has no conflicting owner-signed alternative at any new position, including signatures created earlier.

D. The two abstract services

RAI commits to sets by their roots and agrees on candidates by their digests. Two services turn these commitments into shared state; RAI states their interfaces and the properties it uses, not their mechanics.

Convergence. A reporter calls $\text{Freeze}_i(\text{ctx}_e, T_i, G_i)$, which returns the signed header R_i of Section V-B and retains

the frozen sets. Any validator calls $\text{Reconstruct}_j(R_i)$, which returns (T, G) or is still pending.

C1 (Integrity). If $\text{Reconstruct}_j(R_i)$ returns (T, G) at a correct j , then $\text{Root}(T)$ and $\text{Root}(G)$ are the roots signed in R_i . With collision resistance, $(T, G) = (T_i, G_i)$ when i is correct, and a Byzantine i is bound to one pair per header.

C2 (Termination). If a correct i froze R_i and R_i reaches a correct j , then under the data service and fair retry $\text{Reconstruct}_j(R_i)$ eventually returns.

Full transfer of the committed sets, an authenticated difference against a shared projection, and set reconciliation all satisfy C1 and C2; they differ in cost. A Byzantine reporter can make only its own report unreconstructible.

Consensus. For handoff e , members of the closing committee K_e call $\text{Propose}(\text{ctx}_e, v)$ with candidates $v = (Q, \mu, d)$ of Section V-D, and $\text{Decide}(\text{ctx}_e)$ returns a value v with a proof π_e that $\text{Verify}(\text{ctx}_e, v, \pi_e)$ accepts at any party knowing K_e . RAI supplies the predicate $\text{Eligible}_e(v)$ of Section V-D, which a correct member can evaluate once it has reconstructed the reports v names.

A1 (Agreement). Two accepted proofs for ctx_e carry the same v .

A2 (Eligibility). An accepted proof for ctx_e carries a v with $\text{Eligible}_e(v)$: a correct member supports a value only after checking Eligible_e on reconstructed inputs, and no decision forms without correct support.

A3 (Integrity). A proof cannot be replayed for another context, and Verify needs only K_e and the retained inputs v names.

A4 (Termination). If at least $N - f$ correct members of K_e eventually hold some eligible candidate for ctx_e , then every correct member eventually decides, given recurring prepared windows long enough for the service's own rounds.

A2 is external validity [4] with a predicate the service treats as opaque. C2 feeds A4: convergence makes correct reports usable, usable reports make an eligible candidate constructible at every correct member, and the consensus service then decides it. Nothing in A1–A4 refers to accounts, blocks, or report contents; any Byzantine agreement protocol with external validity and verifiable decisions, leader-based or not, fits the interface. Safety (Section VI) uses C1 and A1–A3 together with the static fault bound, authentication, persistent voting, and the voting rules of Section IV; termination additionally uses C2, A4, the data service, finite preparation, and recurring prepared windows.

IV. ACCOUNT-LOCAL PROCESSING

A. Blocks, operations, and validation

A block is $B = (a, h_p, x, \sigma_a)$: account, parent hash, ordered operation batch, and owner signature, with identifier $h(B) = H(\text{Block}, a, h_p, x)$. The epoch is absent from the identifier, so that a nonfinal omitted block can be retried with new-epoch votes; an empty batch is permitted for owner-authorized branch continuation. The operations are $\text{Send}(b, z)$,

for destination b and positive amount z , and $\text{Receive}(s)$, where $s = (h(B), j)$ identifies a send at operation index j . A send debits its owner; a receive credits only the named destination and requires an explicit proof that the send is already final.

Validation replays the entire unresolved selected path from its finalized anchor, then the proposed batch, checking signatures, consecutive parent slots, amounts, balances, and duplicate receives; it has no application effect, and every finalization repeats the checks and applies only unapplied operations. A finality proof is an account-certificate proof with the evidence needed to verify eligibility, or a verified decided checkpoint that promoted the block. A proof used for epoch- e voting cannot depend on S_e or a later checkpoint, and a receive's send proof must be available before a correct validator supports the receiving path, which makes replay dependencies well founded (Lemma 6).

B. Single-support voting and durable state

An account instance is $(\text{session}, e, a, v)$. A correct validator first-votes at most once in that instance, for the first admissible available block it accepts; there is no second-look support for a rival and no account timeout vote.

Definition 1 (Support, notarization certificate, exclusion witness): A member of K_e supports a block B in an instance if it signed an epoch- e first vote for B or an epoch- e late notarization for B ; a correct validator supports at most one block per instance. A *notarization certificate* $\text{NC}_e(B)$ is a set of q epoch- e first votes for B from distinct members. An *exclusion witness* $\text{XW}_e(B)$ is a set of q epoch- e supports for B from distinct members, first votes or late notarizations in any mix. Every $\text{NC}_e(B)$ is an $\text{XW}_e(B)$; the converse fails.

An NC of first votes is what makes a block complete, what a validator records before final-voting, what an N report entry claims, and what Rules 1 and 3 of Section V-E consume; an XW serves only as the closing-epoch evidence of the overlap exception, as the discharge of a recovery record of origin e on a conflicting block, and inside the overlap certificate behind an F claim. Letting late notarizations count toward an NC makes correct reports unusable (Section VII).

A block is *complete* in epoch e when its data and required ancestry are available and the validator has assembled its NC_e . A complete eligible block is durably recorded as notarized before the validator final-votes, uses it as a complete parent, or freezes its report. The validator can final-vote only for its own first-voted block, and atomically records a terminal flag with that signature.

Certificates and the early-vote rule. Every first vote names its *base*, the last checkpoint its signer had installed when signing; the admission-witness digest binds it. An epoch- e first vote is *settled* if its base is S_{e-1} and *early* otherwise. An eligible block finalizes on an $\text{FC}_e(B)$ of q final votes or an $\text{FF}_e(B)$ of $N - p$ settled first votes; early first votes count toward $\text{NC}_e(B)$ and $\text{XW}_e(B)$ and toward nothing else. A validator that has not installed S_{e-1} releases an epoch- e final vote for B only while it holds, for B and every unresolved block of its selected prefix, the closing-epoch exclusion witness and the

current-epoch NC that the overlap exception requires, and it retains them with its notarization record; after installing S_{e-1} it final-votes under ordinary rechecking. A block first-voted during a close therefore finalizes in two rounds rather than one. Without the rule, a fast certificate can be formed from early votes whose eligibility rests on a witness their signers never held, and the next closure cannot see it (Section VII).

A finalized block's selected unresolved ancestors finalize with it, and incompatible provisional branches leave the effective live view; an NC alone has no balance effect.

Cross-epoch voting record. A correct identity stops epoch- $(e - 1)$ first and final voting before opening epoch e . While S_{e-1} is unknown, it does not first-vote for a different block at a position where it supported a block in epoch $e - 1$; re-voting the same block is allowed. Once the predecessor is installed, its finalized history, frontier, and carried lock records govern the position.

Late notarization. A member of K_{e-1} keeps notarizing for its closing epoch until it installs S_{e-1} . When it first-votes B at (a, v) in epoch e while S_{e-1} is unknown, it also signs a notarization-only epoch- $(e - 1)$ vote for B , unless it supported a different block at (a, v) in epoch $e - 1$; if it first-voted B itself there, that vote already supports B . The late vote is durably recorded before release. It counts toward an epoch- $(e - 1)$ exclusion witness and toward nothing else: no NC, FF, or FC counts it, no final vote and no N entry follows from it, and no report contains it, since reports are frozen before it is cast. Late notarizations are retained evidence that a manifest may carry, but only inside an exclusion witness. They give a block published after the boundary the closing-epoch exclusion witness that the overlap exception requires.

C. Attachment and prefix-wide eligibility

Let $d_a(S)$ be the maximum depth of any block for account a retained in checkpoint S . With S_{e-1} installed, a new epoch- e root starts at depth $d_a(S_{e-1}) + 1$ and extends a maximum-depth retained tip; a retained tip continues at its own depth (Section V-F). Subsequent children require complete epoch- e parents, and an extension must respect finalized history and every carried lock record on its selected ancestry. Before S_{e-1} is known, first voting may prepare work from a maximum-depth tip of the latest closed ledger, from a closing-epoch finalized parent with its proof, or from a complete current-epoch parent. The signed admission-witness digest fixes the anchor and old evidence actually used.

Ordinary early work is provisional: when the predecessor arrives, it is rechecked against the decided frontier, finalized history, and carried lock records, and a tally of fast-final size does not bypass this check. A notarization lock constrains settlement to its branch. A recovery record of origin o on a block B' is *discharged* by a verified exclusion witness $\text{XW}_o(B)$ for a block B that conflicts with B' : account exclusion (Lemma 1) then proves that no epoch- o final certificate for B' can exist. The witness is retained. Evidence of any other epoch does not discharge the record, and a block carrying several records is

released only when each has been discharged by a witness of its own origin.

The core overlap exception. A block may finalize before S_{e-1} is known only if the verifier checks its entire selected prefix against the latest closed state and proves cross-boundary exclusion at *every unresolved position being finalized*: each such block, including the certificate target, must have a closing-epoch exclusion witness XW_{e-1} and a current-epoch NC_e for the same hash. A closing-epoch finalized prefix is already a safe anchor. Any inherited recovery record bypassed by this path requires the matching-origin discharge witness above; inherited notarization locks cannot be bypassed. We call the bundle of these witnesses, the final certificate, and the discharge witnesses for the bypassed records an *overlap certificate* OC. Its validity is a function of its contents and of the last closed state alone (Lemma 7), so a validator that learns it before installing S_{e-1} and one that learns it after reach the same verdict.

A fresh descendant without its own closing-epoch exclusion witness remains provisional, even if its parent is notarized in both epochs; otherwise disjoint committees could certify different children of that parent. If the predecessor arrives first, nonfinal early work falls back to ordinary rechecking; already finalized overlap work remains in the certified live overlay.

Fresh-child recovery. If a checkpoint retains unresolved tips, the owner can extend a permissible tip with a fresh child; finalizing that child selects its unresolved ancestors rather than reopening their slots, and an undischarged lock record still constrains the branch choice. Persistent submission therefore means retrying an omitted block or supplying a fresh, possibly empty, uncontested child of a retained admissible tip.

V. PIPELINED EPOCH TRANSITIONS

A. Lagged committees and local boundaries

The account committee for epoch e is $K_e = C_{e-2}$. That outgoing committee decides S_e ; the already known successor $C_{e-1} = K_{e+1}$ verifies and installs it. The state S_e derives C_e , whose account service starts in epoch $e + 2$. Genesis supplies $S_{-1} = S_G$ and $C_{-2} = C_{-1} = C_G$.

After local duration D , a validator may end epoch e only when no older epoch remains closing. It stops old first and final voting, records all complete eligible old blocks as notarized, freezes its report against the closed S_{e-1} , and opens $e + 1$; learning a decided checkpoint also stops any remaining signing for its closed epoch. A validator signs nothing new in an epoch it has left except late notarizations, which stop when it installs the checkpoint, but re-broadcasts what it signed before, so that certificates of the closing epoch can still complete. If closure outlasts D , the next local boundary is deferred while successor voting continues. Boundaries are local events; nothing requires a synchronized stop.

B. Immutable signed reports

Every correct outgoing validator i signs one header $R_i = \text{Sign}_i(\text{ctx}_e, \text{Root}(T_i), \text{Root}(G_i))$, where the context binds the

session, closing epoch, predecessor checkpoint, outgoing committee, and reporter identity. A Byzantine identity may equivocate, but a candidate selects at most one of its headers. Let L_i be the reporter's ledger at freeze time. Its canonical tagged set is $T_i = \{(h(B), s_i(B)) : B \in L_i\}$ with $s_i(B) \in \{R, N, F\}$: R is inherited recovery-protected state, N represented notarization without locally observed finality, F finality by inherited checkpoint state or by an explicit account certificate for the block or a descendant selecting it; the strongest status wins. Each entry also names the block's account position and parent, so that a selection places every block it names without the block body. Let $V_i^{(e)}$ contain the hashes of all blocks for which i released an epoch- e first vote before freezing. The second committed set is

$$G_i = V_i^{(e)} \setminus \{h : (h, N) \in T_i \text{ or } (h, F) \in T_i\}.$$

An N or F entry already exposes the vote. An R entry does not: it records what the predecessor carried, not what i signed in epoch e , so a reporter that first-voted an R-tagged block lists it in both T_i and G_i . An earlier version removed every key of T_i ; Section VII shows how that hides a fresh vote and lets two conflicting fast certificates form.

Correct reporters retain the frozen sets, vote records, and admission evidence; later events change their live view but not the signed report. Late notarizations are released after freezing, so they are in no $V_i^{(e)}$; they create no N entry, no Rule 1 or Rule 2 support, and no Rule 3 promotion.

C. Usable reports

Reconstruction is provided by the convergence service with integrity C1; RAI adds the semantic check. A report is *usable* at j when both sets are reconstructed, every entry is well formed, and its claims are justified by evidence j holds: an R entry matches predecessor recovery state; an N entry has the relevant NC of q first votes or an inherited notarization record; an F entry has valid inherited checkpoint finality, an explicit account certificate, or an overlap certificate; and a G_i membership is backed by i 's own signed epoch- e first vote and names no hash that T_i tags N or F (it may name one tagged R). A report naming a block above the first position without its parent, unless the predecessor finalized that block there, is malformed and never selected, so one Byzantine report cannot make every candidate unconstructible. Missing evidence is fetched from retained data; a Byzantine report with unavailable or malformed material stays unusable. Closure needs only $N - f$ usable reports, and at least that many reporters are correct.

D. Eligible candidates

A candidate selects exactly $N - f$ usable report headers from distinct members of K_e , in canonical identity order, forming Q , and commits to an evidence manifest $\mathcal{M}$ listing content-addressed blocks, report reconstructions, the exact supporting vote records including the late notarizations inside exclusion witnesses, overlap certificates, and finite admission and dependency witnesses, all of them retained protocol data.

The candidate value is $v_e = (Q, \mu_e, d_e)$ with $\mu_e = H(\mathcal{M})$, $d_e = H(S_e)$, and $S_e = \text{BuildState}(S_{e-1}, Q, \mathcal{M})$. RAI's eligibility predicate, which the consensus service enforces through A2, holds for $v = (Q, \mu, d)$ when Q has $N - f$ usable reports of distinct members of K_e , $H(\mathcal{M}) = \mu$, $\text{BuildState}(S_{e-1}, Q, \mathcal{M}) = S \neq \perp$, and $H(S) = d$, with all contexts, report identities, vote domains, and predecessor references verified. A correct member evaluates Eligible_e only on reconstructed and validated inputs; a digest establishes neither availability nor validity. The manifest fixes the witness bytes before voting, so the state digest is a function of fixed semantic inputs rather than of certificate arrival order; it is not a license to omit required report material, nor may it promote an N or G_i entry to final because an incidental certificate is present.

E. Deterministic construction

A candidate block is an inherited block, a selected tagged block, or a block named by a selected G_i , with the ancestors and fixed dependencies needed to validate it. Eligibility is checked against the closed predecessor or the overlap evidence retained with its admission, never against the candidate's own certificate, which would make Rule 2 circular; an unexplained speculative path is excluded from new recovery counts. For a block B , let $f_Q(B)$ mean that a selected F entry names B and its finality witness verifies, and let the fresh reporter support set $U_Q(B)$ contain the selected reporters i with $h(B) \in G_i$ whose epoch- e first vote for B the manifest verifies. An inherited R entry is not itself support; an epoch- e first vote for an R-tagged block is, and reaches U_Q through G_i .

Final closure. Start with inherited finality and all blocks satisfying f_Q . Take account-ancestor closure and recursively include the explicit finalized-send dependencies required by those final blocks. Call the result F^* .

Rule 1: represented notarization. At each new account position, after excluding conflicts with F^* , retain the unique eligible represented NC target and its ancestry as a notarization lock; same-domain exclusion rules out two conflicting eligible NCs.

Rule 2: reporter first votes. If there is no represented eligible NC at that position and exactly one eligible block has $|U_Q(B)| \geq r$, retain that block and its ancestry under a recovery record of origin e , the closing epoch, whether or not the block already carries records of earlier origins. Theorem 1 shows that a formable fast-final block cannot coexist with a threshold rival.

Inherited state and discharge. Carry every predecessor lock record forward unless explicit compatible finality excludes its block or, for a recovery record of origin o , the manifest contains an exclusion witness $XW_o(\cdot)$ for a conflicting block. A selected vote or claim for a block that conflicts with a carried record is eligible only if the manifest contains that witness; the record is then discharged in this construction. A block leaves the retained state only when none of its records survives and it is not an ancestor of a block with a surviving record. Discharge

Algorithm 1 $\text{BuildState}(S_{e-1}, Q, \mathcal{M})$

- 1: Verify the predecessor, selected identities, report roots, and the committed manifest.
 - 2: Take every selected (T_i, G_i) from Reconstruct; verify the material for all required memberships and claims.
 - 3: Form the parent-closed candidate set; classify admission and eligibility from fixed evidence.
 - 4: Compute F^* by explicit-finality, ancestor, and finalized-send dependency closure.
 - 5: Compute represented NC targets (first votes only) and support sets U_Q from every selected G_i , including members that T_i tags R, for every position before pruning.
 - 6: For each inherited recovery record, look for an exclusion witness of the record's origin in $\mathcal{M}$ and discharge that record only. Verify each F claim that rests on an overlap certificate (Lemma 7). Reject claims that conflict with a surviving record.
 - 7: Apply Rules 1–2; stamp each Rule 2 record with origin e ; retain surviving inherited records and the ancestor closure of required lock targets, forming R_Q .
 - 8: Apply Rule 3 to obtain P_Q ; compute F^+ from $F^* \cup P_Q$.
 - 9: Retain $R_Q \setminus F^+$ and replay $F^+ \setminus F_{e-1}$ once in canonical dependency order.
 - 10: Return (L_e, Σ_e) ; Rule 3 has effect only if this candidate is decided.
-

is never inferred from a certificate's newer epoch number, and all evidence summaries are computed before any omission. Let R_Q be the surviving inherited unresolved state, selected lock targets, and their ancestor closure.

Rule 3: checkpoint promotion. For each account, starting at the tip of F^* , promote the next block only while it is the unique child retained in R_Q and the manifest verifies a closing-epoch NC of first votes for it; stop at the first fork or block without such an NC. The prefix P_Q is candidate-final until the consensus service decides the candidate. Recovery-only blocks are never promoted. Let F^+ be the ancestor- and finalized-send-dependency closure of $F^* \cup P_Q$; the output ledger marks F^+ final and retains $R_Q \setminus F^+$.

Replay orders parents before children and finalized sends before dependent receives, choosing the least $(a, v, h(B))$ among independent ready blocks. A cyclic or semantically invalid graph is rejected; Lemma 8 shows that an all-correct usable selection never is.

F. Installing a decided checkpoint

Let a validator's current closed base be S_b with finalized set F_b , and O_i its certified overlay, so that its effective final set is $J_i = F_b \cup O_i$. On an accepted proof π_e for v_e , the validator recomputes S_e from the inputs v_e names, checks $H(S_e) = d_e$, and installs it with final set F_e : $J'_i = J_i \cup F_e$ and $O'_i = J'_i \setminus F_e$. The union is checked for account compatibility, final dependencies, and valid lock discharge; the validator adopts S_e as its base and materializes the live view as $\text{Replay}(\Sigma_e, O'_i)$, with persistent operation identifiers suppressing duplicate application. The base's final set grows monotonically; every overlay block has an explicit account-finality proof for itself or a selecting descendant; and the union extends each base-final account prefix and respects every undischarged lock record.

Committee C_{e-1} holds no second election: it verifies the outgoing committee's proof, reconstructs the state, derives C_e from the canonical Σ_e , and rechecks nonfinal early work, discarding invalidated provisional work without clearing its persisted first vote or terminal state. Locks the checkpoint retains are continued, not reopened: every retained block gets an instance in the open epoch at its own position, and a rival at that position is refused while any record on the retained block survives. A first vote in that instance is an ordinary epoch- e first vote; if the block does not finalize, the vote is exposed at the next closure through G_i , and Rule 2 gives the block a record of origin e .

VI. CORRECTNESS

The arguments concern the core protocol, including the prefix-wide overlap guard, late notarization, the early-vote rule, origin-stamped records, and Rule 3 promotion, and are relative to the services of Section III-D: safety uses C1 and A1–A3, termination additionally C2 and A4. Cryptographic guarantees hold except with negligible probability. We state the results with the arguments that carry the weight; the supplementary material has the full proofs.

A. Safety

Lemma 1 (Account exclusion): In one account instance at most one block has an exclusion witness; in particular at most one has an NC. If an FC or FF for B can be formed in that instance, no conflicting block has an exclusion witness. If an FF for B can be formed, every conflicting block has at most $f + p < r$ supporters.

Proof: Two sets of q supporters intersect in $2q - N = f + 1$ identities, one correct, and a correct validator supports one block per instance (Definition 1). A correct final voter first-voted the same block, so an FC intersects a rival's XW the same way; an FF has $N - p \geq q$ first voters, and with $f' \leq f$ actual Byzantine members at least $N - p - f'$ correct ones, leaving at most $p + f'$ supporters for a rival. ■

Lemma 2 (Late notarization adds no old finality): Every epoch- e NC, FC, or FF consists of first and final votes released by correct signers before they left epoch e , together with Byzantine votes. A correct validator releases a final vote, or records an N entry, only for a block for which it holds an NC of q first votes.

Lemma 3 (Frozen-report exposure): Let i be a correct reporter for closure e . If i released an epoch- e final vote for B , then T_i tags B with N or F. If i released an epoch- e first vote for B , then T_i tags B with N or F, or $h(B) \in G_i$, whatever tag B inherits from S_{e-1} . Both persist after the report is frozen.

Lemma 4 (Signers hold the witness): Every correct signer of an eligible epoch- e FC or FF for B holds, as retained evidence, an exclusion witness discharging each recovery record of S_{e-1} on a block that conflicts with B .

Proof: An FF consists of settled first votes, whose correct signers had installed S_{e-1} and were refused a rival of a retained block unless every record on it was discharged. A correct final voter either voted after installing S_{e-1} , with the

same check, or before, holding $XW_{e-1}(B)$ by the early-vote rule, which discharges every record closure $e - 1$ created; the records S_{e-1} inherited were known to the voter, and the overlap guard required their matching-origin witnesses. ■

Definition 2 (Closure invariant): A closed state S_e satisfies the closure invariant if, for every epoch $e' \leq e$ and every block B with an eligible epoch- e' account final certificate that can be formed from released signatures, S_e holds B (i) as final, (ii) under a notarization lock, or (iii) under a recovery record of origin e' , directly or as an ancestor of a block so held.

Theorem 1 (Closure preserves latent and late finality): Let S_{e-1} be a valid closed predecessor satisfying the closure invariant, and let v be any candidate with $\text{Eligible}_e(v)$ and state S_e . If an eligible epoch- e account final certificate for B can be formed, possibly completed after reports froze or after the checkpoint was decided, then S_e holds B as in (i)–(iii) with $e' = e$, no admissible evidence discharges that hold, and S_e satisfies the closure invariant.

Proof: Let Q be the selected reporters and A the certificate's signers. If B conflicts with a carried record, each correct signer holds the discharging witness (Lemma 4); it is retained evidence, the manifest contains it, and line 6 of Algorithm 1 discharges the record, so B is eligible for Rules 1–2 whenever a correct signer is selected. For a normal certificate, $|(Q \cap A) \setminus \text{Byz}| \geq q - 2f = p + 1$, and each such reporter froze an N or F entry for B (Lemma 3): a verified F entry gives (i), otherwise B is the unique represented NC target at its position (Lemma 1) and Rule 1 gives (ii). For a fast certificate, $|(Q \cap A) \setminus \text{Byz}| \geq N - p - 2f = r$; if none of these reporters has an N or F entry for B , each has $h(B) \in G_i$ with a verified first vote, whether or not T_i tags B with R, so $|U_Q(B)| \geq r$, while every rival has fewer than r supporters and no represented NC (Lemma 1); Rule 2 gives (iii). A record of origin e on B is discharged only by an XW_e of a rival, excluded by Lemma 1; holds of earlier origins are inherited, since dropping one needs a conflicting final block or an $XW_{e'}$ of a rival, both excluded. Only $A \cap Q$ is used, so it does not matter when the certificate is assembled. ■

Lemma 5 (Checkpoint-promotion safety): A Rule 3 promoted block cannot conflict with a block that has a closing-epoch exclusion witness, FC, or FF; a decision that finalizes the promoted prefix is compatible with old-domain and core overlap finality.

Lemma 6 (Acyclic final dependencies): For any finite set of valid final blocks closed under account ancestors and required finalized sends, parent and send-to-receive edges form a directed acyclic graph.

Lemma 7 (Overlap verdict independent of installation order): Let closure e be in progress, let B and B' conflict at one position, and let OC be an overlap certificate for B : an $XW_e(B)$, an $NC_{e+1}(B)$, an epoch- $(e + 1)$ FC or FF for B or a descendant selecting it, and a matching-origin exclusion witness for every recovery record on B' in S_{e-1} , with no notarization lock on B' there. Then every candidate with Eligible_e and state S_e (a) does not finalize B' , (b) carries no notarization lock on B' , and (c) carries on B' only recovery

records discharged by witnesses in OC. Hence a validator holding OC accepts B before installing S_e if and only if it accepts B by rechecking after, and a later manifest containing OC verifies an F claim for B against S_e .

Proof: An epoch- e certificate for B' , or the $\text{NC}_e(B')$ that Rule 1 or Rule 3 needs, is excluded by $\text{XW}_e(B)$ and Lemma 1; inherited finality or a notarization lock on B' contradicts the check of OC against S_{e-1} . A record on B' in S_e has origin e , discharged by $\text{XW}_e(B) \subseteq \text{OC}$, or is inherited, discharged by hypothesis. Rechecking after installation tests exactly these conditions, as does line 6 at closure $e + 1$. The prefix-wide guard applies the lemma at the first divergent unresolved position. ■

Lemma 8 (Deterministic construction): If the predecessor satisfies the closure invariant then, for fixed usable selected reports and a valid immutable manifest, `BuildState` returns one canonical candidate state at every correct validator, preserving inherited finality and all mandatory surviving records and applying only explicit account finality plus Rule 3 promotion. All-correct usable inputs yield a finite valid construction. The proof uses C1 for identical inputs, compares records as sets of (block, strength, origin), and relies on Lemma 7 for line 6 to accept a correct reporter’s F claim for an overlap-final block.

Theorem 2 (Transfer safety across epochs): Assuming C1 and A1–A3, finalized histories at correct validators are compatible and irreversible across all core-protocol epochs; finalized balances are nonnegative, and each send is received at most once.

Proof: Induct from genesis. Within an epoch, Lemma 1 excludes incompatible account finality. At a boundary, A2 makes the decided v_e eligible, so Theorem 1 preserves every formable old final branch and Lemma 5 makes its promotion compatible; A1 and A3 make every correct validator install the same v_e , hence by Lemma 8 the same state. For overlap, the first divergent unresolved position on an early successor-final prefix has a closing-epoch exclusion witness for the successor block, whose late notarizations add no old finality (Lemma 2); a conflicting closing-epoch certificate or Rule 3 promotion would need a conflicting NC, contradicting Lemma 1; and by Lemma 7 correct validators holding the same overlap certificate reach the same verdict before or after installing S_e , so they agree on which overlap blocks are final. The base/overlay union with applied-operation identifiers prevents rollback and duplicate application. ■

The proof uses nothing about how the consensus service decides, since Theorem 1 is universal over eligible candidates.

B. Conditional termination

Lemma 9 (Correct reports become usable): Under the data service, finite relevant admitted work, fair processing, and C2, every correct report eventually becomes usable at every correct validator.

A correct report is finite and well formed (Lemma 2); C2 reconstructs it and the data service delivers the evidence

behind its claims. A Byzantine reporter can make only its own report unusable.

Theorem 3 (Handoff termination): Assume the data service, fair processing, C2, A4, and recurring prepared windows with $N - f$ responsive correct members of K_e long enough for reconstruction, candidate preparation, and the consensus service’s rounds. Every enabled handoff is eventually decided and, with eventual retained-data delivery, installed by correct successors.

By Lemma 9 every correct member constructs an eligible candidate (Lemma 8), A4 decides one, A3 lets successors verify it, and retained data let them install it.

Theorem 4 (Uncontested account continuation): Assume handoff termination, available final-send dependencies, and recurring common-open ready intervals. An owner following the retry-or-fresh-child policy of Section IV eventually finalizes each target of a finite valid uncontested continuation, provided each required new position has such an interval, and the finalized targets eventually enter a checkpoint.

In a ready interval, $N - f \geq q$ correct first votes create an NC and the same validators final-vote; an attempt interrupted by a boundary is preserved by Theorem 1, an omitted block can be retried, and a remaining retained fork can be selected by a fresh uncontested child. Normal finality takes three delivery stages after broadcast and an $N - p$ settled first-vote quorum two; a sufficient account interval budget is $D/(1 + \rho) > \kappa_A + R_A + 3\delta$, where κ_A bounds opening skew and R_A readiness after the last opening, including predecessor closure. These are conditional communication bounds, not measured latency.

VII. MODEL CHECKING THE HANDOFF

Three rules of Sections IV and V replaced earlier rules that written arguments had accepted: the second committed set G_i (Fix B), discharge by an exclusion witness of matching origin with the overlap certificate, and the early-vote rule. We wrote three TLA^+ models of the handoff and checked them with TLC. Each takes one account position with two conflicting blocks, $N = 6$, $f = p = 1$, five correct validators and one Byzantine validator whose signatures are available for every block, vote kind, and epoch and whose report is chosen adversarially subject to the usability checks. The consensus service is the nondeterministic choice of one eligible candidate; TLC explores all interleavings, with validators identified up to permutation. The models, configurations, and TLC outputs with the violating traces are in the supplementary material.

`RaiClose` has three voting epochs and two closures, no early voting, and `BuildState` with Rules 1–3, inherited records, and discharge; its invariants are transfer safety at the position, Theorem 1 without any hypothesis about witnesses, and one final block per checkpoint. `RaiOverlap` has one closing and one open epoch: validators cross their boundary one by one, first-vote early under the cross-epoch guard, late-notarize, finalize under the overlap exception or by rechecking after installation, and one closure builds S_1 ; its invariants add the same verdict before and after installation (Lemma 7), usability of every correct report (Lemma 9), and admissibility of the

TABLE II
TLC RESULTS. DISTINCT STATES FOR COMPLETED RUNS.

Model and switches	Result	States
Close, old G_i	Thm. 1 fails (19 steps); two FFs (23)	–
Close, Fix B	no violation	651,632
Overlap, all repairs	no violation	5,217,510
Overlap, no discharge	same verdict fails (15)	–
Overlap, no late evidence	witness admissible fails (14)	–
Overlap, late votes in NC	usable reports fails (9); safety holds	5,485,444
Handoff, same K , old FF	Thm. 1 fails for S_2 (22)	–
Handoff, disjoint K , old FF	Thm. 1 fails for S_2 (23)	–
Handoff, same K , rule	no violation	8,347,138
Handoff, disjoint K , rule	no violation	3,707,876
Handoff, 3 shared, rule	no violation	121,053,272

discharge witness at the next closure, and three switches turn the repairs off. `RaiHandoff` combines early voting, two closures, and a changing committee, with the Byzantine validator in both committees; a discharge witness reaches the builder of S_2 only if a selected correct reporter relied on it, which models a withheld signature.

Table II gives the results. With the earlier $G_i = V_i^{(e)} \setminus \text{keys}(T_i)$, a reporter that first-voted a block the predecessor carries under a recovery record tags it R, the fresh vote appears nowhere, and two conflicting fast certificates are reached. Without discharge, two correct validators finalize different blocks depending on installation order; without late notarizations as evidence, the next closure cannot verify a correct F claim; with late notarizations counting toward an NC, a correct report becomes unusable, which costs termination but not safety. With the fast path counting every first vote, Theorem 1 fails for the second closure with identical and with disjoint committees: four early votes for B are eligible only through a witness that one unselected validator holds, so S_2 retains the rival, which a later epoch finalizes while B 's fast certificate can still be formed. With the repairs the invariants hold in every run, and probes confirm that a block can still finalize before S_1 is decided, that the second closure does discharge first-checkpoint records, and that a block can carry records of two origins.

No model covers more than one account position (so ancestor closure, the prefix-wide guard, and fresh-child recovery), a third epoch, receive dependencies, crash and restart, other committees, or any N other than 6; each model's abstractions only add behaviours.

VIII. COSTS, SCOPE, AND LIMITATIONS

The agreed value contains $N - f$ report headers and two digests, so its reference portion is $O(N)$, independent of the number of account blocks; the manifest, reconstruction, validation, and retained histories scale with old state, new

votes, and unresolved branches. A correct validator releases at most one first vote, one final vote, and, during a close, one late notarization per account instance; this bounds neither an adversary's owner-signed alternatives nor traffic or validation work.

The pipeline has an explicit frontier. A block whose late notarizations split, or that lacks one of the q supporters, waits for the predecessor checkpoint; a block first-voted during a close finalizes in two rounds, since early votes form no fast certificate; and a block whose closing-epoch exclusion witness is withheld cannot become final behind the checkpoint. Checkpoint promotion finalizes a uniquely retained closing-epoch notarized prefix without an account certificate, never recovery-only state, and resolves no fork: a position whose votes split evenly stays retained, identically everywhere, until the owner extends it or a later epoch's exclusion witness discharges the records of one side.

Forks and retained dependencies can accumulate without bound, and lock records, late notarizations, and overlap certificates must be kept as long as a record they can discharge survives; the model supplies no admission-control or garbage-collection policy. RAI is for authenticated committees with an externally maintained fault bound, not a permissionless staking system; the static adversary excludes later compromise of departed validators, and account operations exclude shared objects, multiwriter transactions, and a guaranteed winner under arbitrary equivocation.

IX. RELATED WORK

Guerraoui et al. characterize asset transfer's consensus number, including the single-owner case [1]. FastPay provides prefunded Byzantine settlement without a global transaction order [2]; Astro builds asynchronous payments on broadcast [3]; Zef adds private coin payments [5]; Nano provides account-local chains with distinct send and receive operations [6]. These establish the application-level opportunity, not RAI's transition rules. Astro's extended version discusses asynchronous replica reconfiguration with view-tagged messages and account-log transfer [3], [7]; RAI's distinction is the frozen-report evidence that preserves first- and final-vote obligations across a boundary while the next epoch is already voting.

Sui Lutriss uses consensus for shared objects and checkpoint sequencing, including end-of-epoch handling of processed certificates [8]; RAI does not continuously sequence ordinary certificates, so there is no certificate sequence to drain before the epoch ends. Cuttlefish broadens the fast path with collective objects and FastUnlock [9]; RAI's boundary promotion is narrower. Pastro studies permissionless asynchronous asset transfer with weighted stake and a dynamic adversary [10]; Dynamic Byzantine Reliable Broadcast and asynchronous Byzantine reconfiguration address broader changing-participation settings [7], [11]. PDCC combines certificate-based payments with membership change and periodic settlement; its installation rule accepts certificates for the installed configuration and states a correct-member overlap assumption [12], whereas RAI

preserves eligible old-domain certificates assembled later and needs no members common to consecutive committees.

Consensus with external validity is a standard interface [4]; RAI uses it with the eligibility predicate of Section V-D and instantiates it with a leader-based voting construction [13], [14]. Authenticated set reconciliation is a mature building block [15], [16], [17]. Model checking small configurations of Byzantine protocols with TLC is common practice [18]; here it served as a regression test for the handoff rules rather than as a proof.

X. IMPLEMENTATION AND EVALUATION

Prototype. We implemented RAI in RsNano [19], a Rust implementation of the Nano node [6]. Account elections follow Section IV with the equal-weight thresholds of Table I ($f = p = 1$, $N = 6$), late notarization, the early-vote rule, and the prefix-wide overlap rule; checkpoints carry origin-stamped records and are built by Algorithm 1 with Rule 3 promotion and matching-origin discharge. Epochs end at fixed times $T_0 + kD$ on every validator’s local clock. Every first vote, final vote, late notarization, left epoch, frozen report, and overlap certificate is written to an LMDB environment of its own before its signature leaves the process, and a restarted node reloads these records and refuses whatever they forbid. Late notarizations open no account instance at their receivers and are kept apart from the vote records that reports are built from; a manifest carries them, with the settled bit of every first vote, as bit sets over the committee. A position whose first votes are already split so that no block can reach q is not re-voted at the boundary.

Convergence by rateless set reconciliation. A report’s two committed sets are reconstructed with rateless invertible Bloom lookup tables [15], [16], [17]. Each set element is a fixed 105-byte item naming account, height, hash, parent, and tag. A validator that needs a report starts from its own live projection of the closing epoch, which usually differs from the reporter’s frozen set in a few entries, and asks any holder of the reconstructed set for coded symbols in growing batches; peeling them against the local projection yields the symmetric difference, which the requester applies and checks against the signed root (C1). It then fetches, by hash, the signed votes behind every N and F entry and every residual record it does not already hold. *Consensus by a Kudzu-style election.* The closing committee decides the checkpoint with one leader-based voting instance that adapts Kudzu’s voting construction [14]: first votes, second-look notarization support, timeout votes, and certificates with thresholds q (normal), $N - p$ (fast), and q timeout votes to skip a slot. Election votes and account votes are separate domains, and every election signature binds the handoff context, the deciding committee, the slot, and the proposal hash or timeout value. Four points differ from Kudzu and are what makes the instance provide A1–A4. There is one instance per handoff, not a chain: only a child of the genesis slot may introduce a fresh payload (Q, μ_e, d_e) and a proposal on a certified parent copies its parent’s payload, which with

Kudzu’s exclusion of a conflicting certificate in a decided slot gives A1. Validity is RAI’s eligibility: a validator first-votes a proposal only after reconstructing its reports, checking every manifest claim, running BuildState, and matching d_e (A2), so a final quorum contains correct validators that validated the fixed evidence, which with context binding gives A3. The outgoing committee decides and the successor only verifies the decision proof. Kudzu’s erasure-coded payload distribution is not used and its communication bound is not claimed. Leaders rotate in canonical key order and propose the candidate built from the canonical prefix of usable reports; a validator whose own votes yield μ_e has checked that evidence already, and otherwise fetches the manifest from the proposer. A slot whose leader is offline or Byzantine times out and the next leader proposes. The timeout is local: a validator waits up to three times longer while it is still checking the leader’s proposal, or while the leader is a usable reporter whose proposal has not arrived, since a validator that times out cannot vote for the proposal once checked; a later leader proposes again a payload it already validated, which validators that checked it accept at once. Under recurring prepared windows with $N - f$ responsive correct members and a local timeout satisfying $\Delta_E / (1 + \rho) > \kappa_E + \tau_E + \delta$, where κ_E bounds slot-entry skew and τ_E bounds proposal preparation and validation [20], Kudzu’s progress argument gives A4.

Departures from the protocol. The manifest commits to signed votes; blocks and ancestry are placed by the report items, and admission witnesses are not committed separately. The close election’s own votes are not persisted. Retained vote records keep a cap of four epochs plus whatever may still discharge a surviving record. A first vote names its base by one signed bit, settled or early, since S_{e-1} is unique. The restart path is exercised by unit tests only. The supplementary material lists the defects found while measuring.

Baseline and setup. The baseline is unmodified RsNano (develop), which runs Nano’s open representative voting: a block is confirmed once final votes reach 67% of the online weight, after a round of non-final votes; it has no epochs and no checkpoints, and since final votes cannot be withdrawn, a fork whose votes split can leave no side able to reach the quorum. All validators run as separate processes on one MacBook Air (Apple M1, eight cores, 16 GiB) over loopback; LMDB runs without synchronous writes. Six equal-weight representatives hold the voting weight. A load generator funds 45,000 accounts and then publishes 45,000 blocks at an offered 2,000 blocks/s, each on the frontier of a randomly chosen account, following the owner policy of Section IV: it builds on a confirmed frontier or, once an unconfirmed non-fork frontier has waited 5 s, extends it with a fresh child, and never extends a position where it published a fork. With forks enabled it publishes a conflicting sibling for 5% or 10% of the blocks, each sibling to half of the validators. An *offline* validator runs no node but keeps its weight; a *Byzantine* validator also runs no node, and the generator votes with its key at random, choosing any vote kind for recent blocks and contradicting itself. RAI

TABLE III

NON-FORK GOODPUT (BLOCKS/S) AND NON-FORK CONFIRMATION LATENCY (MS) AT AN OFFERED 2,000 BLOCKS/S, ONE RUN PER VARIANT. CLOSE IS THE LONGEST CLOSE (S), OPEN THE NUMBER OF POSITIONS STILL UNRESOLVED WHEN THE RUN SETTLED (IDENTICAL ON EVERY VALIDATOR), CP THE NUMBER OF CHECKPOINTS EVERY RUNNING VALIDATOR DECIDED IDENTICALLY. A BASELINE RUN THAT DID NOT FINISH IS GIVEN BY THE SHARE OF THE 45,000 BLOCKS CONFIRMED AT ITS PUBLISHING DEADLINE.

Variant	RsNano develop (no checkpoints)				RAI prototype (8 s epochs)						
	Blocks/s	p50	p95	p99	Blocks/s	p50	p95	p99	Close	Open	CP
No forks	1901	189	410	622	1,921	118	394	803	4.5	0	3
No forks, 1 offline	1907	207	238	285	1,958	112	281	371	4.0	0	3
No forks, 1 Byzantine	1921	205	234	289	1,957	112	265	513	3.7	0	3
5% forks	did not finish: 95.4% confirmed				1,667	225	1000	2349	8.2	0	5
5% forks, 1 offline	did not finish: 90.4% confirmed				1,592	186	1068	4014	8.6	1,433	3
5% forks, 1 Byzantine	did not finish: 93.0% confirmed				1,567	162	1246	1479	9.0	0	5
10% forks	did not finish: 87.3% confirmed				1,484	406	3141	3717	10.1	0	4
10% forks, 1 offline	did not finish: 69.2% confirmed				1,292	523	2674	3293	14.8	2,920	3
10% forks, 1 Byzantine	did not finish: 86.4% confirmed				1,367	618	3560	3957	15.1	0	6

uses 8 s epochs. A run *settles* when every running validator holds the same cemented state and, for RAI, has decided the same checkpoints; a position left retained by a split vote counts as settled when it is uncemented on every validator alike. We report non-fork blocks only: goodput is the number of non-fork blocks confirmed divided by the publishing-to-last-confirmation time, and latency runs from publication to confirmation at one validator. Each variant ran once per system on an otherwise idle machine. The baseline rows were taken in an earlier pass on the same host with unchanged baseline code; the RAI rows in one pass on the final prototype commit.

Without forks. Table III shows both systems keeping up with the offered load. RAI’s median latency is 112 to 118 ms against 189 to 207 ms: an uncontested block finalizes on one round of $N - p$ settled first votes, where the baseline needs a round of final votes after a quorum of non-final ones. With one validator offline or Byzantine RAI’s tails are close to the baseline’s (p95 265 to 281 ms against 234 to 238); with all six online its p95 of 394 ms came from one close that took 4.5 s, during which blocks one supporter short of a witness waited. Closes otherwise took 1.0 to 2.7 s in the first election round, or 3.7 to 4.0 s when leader rotation reached the missing validator and a second round was needed. In every no-fork variant 89 to 98% of the blocks late-notarized during a close reached a closing-epoch exclusion witness and finalized before the checkpoint.

With forks. The baseline finished none of the six fork variants: forks whose votes split stay unresolved and occupy election slots, which delays the non-fork blocks queued behind them. RAI settled all six with every running validator deciding the same checkpoints, three to six of them, since a close that outlasts the epoch defers the next boundary. At 5% forks the median rose to 162 to 225 ms and p95 to 1.0 to 1.2 s, and the longest close to 8.2 to 9.0 s. Most forked positions were resolved by the time the runs settled, about a third of them by the sibling, each split decided by a matching-origin discharge or by the continuation of the retained block in the next epoch; with one validator offline, 1,433 positions at 5% forks and 2,920 at 10% stayed retained, identically on every

validator. At 10% forks the medians reached 406 to 618 ms and p95 2.7 to 3.6 s. Forks lengthen the closes, since reports, manifests, and discharge witnesses grow with the retained positions: the longest close took 10 s with all validators correct and 15 s with one offline or Byzantine, where a round led by the missing validator costs the timeout and, with a Byzantine member, every proposal carries a manifest the validators must fetch and check; with the timeout rules above no corrected round failed. Late-notarized blocks reached their closing-epoch witness in 86 to 89% of cases at 5% forks and 81 to 83% at 10%; the rest wait for the checkpoint.

Limitations. Each variant ran once. An earlier pass the same day without the timeout rules gave the same medians within noise and, for 10% forks with a Byzantine validator, closes of 17 to 21 s with up to four rounds, p95 3.8 to 5.8 s, and goodput 1,050 to 1,086 blocks/s over five runs; whether split positions end up resolved or retained varied between passes. All validators share one host, so CPU contention rather than network latency sets the delays. Committees are identical across epochs in this matrix; the supplementary material reports a run with per-epoch reweighting but no change of identities. The departures above mean these results validate neither the consensus instantiation’s durability nor a manifest covering admission witnesses.

XI. CONCLUSION

RAI makes epoch transitions for account-local payments pipelined and safe: the next epoch’s voting starts at each validator’s local boundary while the previous epoch closes, immutable reports preserve latent and late finality, a deterministic construction produces a checkpoint with origin-stamped locks, and a predecessor gate with a prefix-wide overlap exception, fed by late notarizations and guarded by the early-vote rule, keeps successor finality compatible with whatever the outgoing committee could still certify. Safety and termination are relative to two abstract services, and model checking found the three failures that written arguments had missed. A prototype settles all nine workload variants with non-fork median latency near 100 ms; a missing member lengthens the closes.

AI TOOL USAGE

ChatGPT assisted with an earlier restructuring of the manuscript, source checking, protocol review, and proof exposition. Claude Code assisted with the prototype implementation, the benchmark harness, running the experiments, diagnosing the defects of Section X, and drafting this revision. The TLA⁺ models were written with the help of ChatGPT and checked with TLC. The proof arguments were reviewed by the authors and have not been machine checked beyond the finite configurations of Section VII.

REFERENCES

- [1] R. Guerraoui, P. Kuznetsov, M. Monti, M. Pavlovic, and D.-A. Seredinschi, “The consensus number of a cryptocurrency,” in *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 307–316.
- [2] M. Baudet, G. Danezis, and A. Sonnino, “FastPay: High-performance byzantine fault tolerant settlement,” in *Proceedings of the ACM Conference on Advances in Financial Technologies (AFT)*, 2020.
- [3] D. Collins, R. Guerraoui, J. Komatovic, M. Monti, A. Xygis, M. Pavlovic, P. Kuznetsov, Y.-A. Pignolet, D.-A. Seredinschi, and A. Tonkikh, “Online payments by merely broadcasting messages (extended version),” 2020, extended version of the DSN 2020 paper; Appendix A describes asynchronous reconfiguration.
- [4] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, “Secure and efficient asynchronous broadcast protocols,” in *Advances in Cryptology – CRYPTO 2001*, ser. LNCS, vol. 2139. Springer, 2001, pp. 524–541.
- [5] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, “Zef: Low-latency, scalable, private payments,” in *Proceedings of the Workshop on Privacy in the Electronic Society (WPES)*, 2023, pp. 1–16.
- [6] C. LeMahieu, “Nano: A feeless distributed cryptocurrency network,” White paper. [Online]. Available: https://content.nano.org/whitepaper/Nano_Whitepaper_en.pdf
- [7] R. Guerraoui, J. Komatovic, P. Kuznetsov, Y.-A. Pignolet, D.-A. Seredinschi, and A. Tonkikh, “Dynamic byzantine reliable broadcast,” 2020, extended technical report, version 2.
- [8] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, “Sui lutris: A blockchain combining broadcast and consensus,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024, pp. 2606–2620.
- [9] L. Kokoris-Kogias, A. Sonnino, and G. Danezis, “Cuttlefish: Expressive fast path blockchains with FastUnlock,” 2023, version 1.
- [10] P. Kuznetsov, Y.-A. Pignolet, P. Ponomarev, and A. Tonkikh, “Permissionless and asynchronous asset transfer,” in *35th International Symposium on Distributed Computing (DISC 2021)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 209. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 28:1–28:19.
- [11] P. Kuznetsov and A. Tonkikh, “Asynchronous reconfiguration with byzantine failures,” *Distributed Computing*, vol. 35, pp. 477–502, 2022.
- [12] L. Shangguan, “Transaction-driven dynamic reconfiguration for certificate-based payment systems,” 2026, initial preprint, version 1, January 14, 2026.
- [13] V. Shoup, “Sing a song of simplex,” in *38th International Symposium on Distributed Computing (DISC 2024)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 319. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 37:1–37:22.
- [14] V. Shoup, J. Sliwinski, and Y. Vonlanthen, “Kudzu: Fast and simple high-throughput BFT,” in *39th International Symposium on Distributed Computing (DISC 2025)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 356. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 42:1–42:19.
- [15] M. T. Goodrich and M. Mitzenmacher, “Invertible bloom lookup tables,” in *49th Annual Allerton Conference on Communication, Control, and Computing*, 2011, pp. 792–799.
- [16] D. Eppstein, M. T. Goodrich, F. Uyeda, and G. Varghese, “What’s the difference? efficient set reconciliation without prior context,” in *Proceedings of the ACM SIGCOMM 2011 Conference*, 2011, pp. 218–229.
- [17] L. Yang, Y. Gilad, and M. Alizadeh, “Practical rateless set reconciliation,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 595–612.
- [18] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [19] RsNano contributors, “Rsnano software repository,” Software repository, base of the RAI prototype and of the evaluated baseline. [Online]. Available: <https://github.com/rsnano-node/rsnano-node>
- [20] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.